

Projet Xplain

Documentation technique

Versions

28/12/2024 **1.0** Création du document et rédaction des points majeures

Table des matières

Table des matières.....	2
Introduction.....	3
Résumé du sujet.....	3
Besoins principaux.....	3
Besoins secondaires et esthétisme.....	3
Besoins secondaires.....	3
Interface graphique.....	4
Page d'historique.....	4
En-tête.....	4
Corps.....	4
Page de conversation.....	5
En-tête.....	5
Corps.....	5
Pied de page.....	5
Détails de productions.....	5
Technologies utilisées.....	6
Technologies Imposées.....	6
Backend :.....	6
Frontend :.....	6
Technologies Complémentaires.....	6
Connexion du Frontend - Quarkus Quinoa.....	7
Streaming de Données - WebSocket.....	8
Gestion de la Base de Données - Agroal.....	9
Téléchargement des modèles - Maven-antrun-plugin.....	10
Messages LLM côté Front - Prisms et Marked.....	11
Répartition des tâches.....	12
Max TEKPA - Backend, communication et direction de production.....	12
Marc LE COQUIL - Frontend, mode natif et traduction.....	12
Travail à deux - Tests, rédaction des documents et recherche dans la documentation technique.....	13
Difficultés rencontrées.....	13
Ce qui a été difficile à réaliser.....	13
Ce qui n'a put être réalisé.....	15
Une documentation contradictoire et éclatée.....	15
Des erreurs non documentées ou aux solutions non fonctionnelles.....	15
Un apprentissage sur le tas.....	17
Des délais trop courts.....	18
Les bugs restants.....	19
Conclusion.....	19
Annexe.....	20

Introduction

Bonjour et bienvenue dans ce document dédié aux développeurs. Nous exposerons ici notre compréhension du sujet et comment nous y avons répondu en amenant progressivement la partie technique de sa réalisation. Nous pointerons aussi l'organisation de notre travail et les difficultés rencontrées par les membres du binôme. Nous passons maintenant au résumé du sujet, nous vous souhaitons une agréable lecture.

Résumé du sujet

Besoins principaux

Ce projet répond au besoin suivant : obtenir l'aide d'un ou plusieurs modèles d'intelligences artificielles (ou Large Language Model (LLM)) en vue de comprendre et corriger des erreurs survenant lors de la compilation de classe Java. Chaque classe, une fois corrigée, démarre une discussion avec le modèle choisi, et chaque discussion ainsi créée vient s'ajouter à une liste sauvegardée et accessible dans une forme d'historique. Il est alors possible de reprendre une ancienne conversation.

L'historique et l'écran de conversation sont sur des pages différentes. De plus nous proposerons au moins trois modèles d'IA dont un lourd et puissant et un léger et moins précis. Avec ces différentes spécifications, nous pouvons passer aux besoins secondaires et à la demande graphique.

Besoins secondaires et esthétisme

Besoin secondaire

Nous avons aussi remarqué que de simples fonctionnalités peuvent être ajoutées à ce besoin principal. Par exemple, il faut que plusieurs fenêtres en même temps puissent corriger leurs classes en même temps. On peut aussi vouloir éditer directement un ancien message plutôt que de devoir le réécrire en entier, et pourquoi pas être en mesure d'éditer un message de correction ? Les

noms par défaut des conversations peuvent être peu digestes, il serait aussi intéressant de les rendre éditables. Et les conversations s'ajouteront avec le temps, il peut être relativement indispensable de penser à un moyen de les supprimer facilement. En parlant de grande quantité de conversations, un outil de recherche parmi elles devient aussi rapidement essentiel à l'ergonomie de l'application. Ainsi, après avoir formalisé tous ces besoins d'importances variables et circonstanciels, nous pouvons penser à l'interface.

Interface graphique

L'interface graphique correspond à tout ce qui est visible, notre application ne comporte aucune dimension sonore donc tout détails manipulable par l'utilisateur se fera au travers de clic souris et interaction clavier.

Page d'historique

La page historique contiendra tout les outils nécessaires à la visualisation et la manipulation de ce dernier pour respecter les besoin exprimés précédemment.

En-tête

L'en-tête devra contenir un titre indicateur de la page, mais aussi un bouton de démarrage d'une nouvelle discussion. Ceux-ci devront être gros et visibles afin de bien situer l'utilisateur. On y ajoutera aussi une barre de recherche pour les conversations.

Corps

Le corps contiendra naturellement chaque conversation à la manière d'une liste triée dans l'ordre anti-chronologique (de la plus récente à la plus ancienne). Cette liste sera défilable et s'étendra sur 3 colonnes afin d'optimiser l'espace sans surcharger l'écran. Enfin, chaque conversation aura un nom éditable au moyen d'un bouton, d'une description qui, par défaut, sera les 3 premières lignes de la première explication de Marx et d'un bouton de suppression de la conversation.

Page de conversation

Cette page se dédie à une unique conversation entre l'utilisateur et Marx. Elle est optimisée pour offrir une expérience ergonomique et naturelle en s'inspirant des modèle de chat bot déjà existant.

En-tête

L'en-tête contient ici deux boutons, celui du retour à l'historique pour plus de commodité pour revenir à ce dernier, et le bouton de changement de modèle de LLM. Ce dernier indique le courant, en cliquant dessus on voit une liste déroulante de ceux proposées par l'application, cliquer sur l'un le met immédiatement en activité.

Corps

Le corps n'est que la liste des interactions. Une interaction est typiquement constituée d'une classe envoyée par l'utilisateur, suivie d'une réponse du compilateur donnant les message de compilation, et enfin la correction de Marx en fonction du LLM choisi. Chaque message précise par qui il est envoyé, quel jour et quelle heure et est accompagné d'une photo de profil pour un visuel plus clair. Chaque message propose aussi un bouton d'édition pour reprendre son contenu directement.

Pied de page

Le pied de page est ici constitué d'un champ de texte invitant l'utilisateur à insérer sa classe. Une fois celle-ci ajoutée, un bouton d'envoi est présent pour lancer l'interaction. Le champ de texte est vidé une fois l'envoi effectué.

Détails de productions

Cette partie se dédie à la production de ce projet. Nous exposerons ici dans un premier temps quelles technologies ont été utilisées et comment, puis nous enchaînerons sur la répartition des tâches au sein du binôme. Nous terminerons par les difficultés rencontrées.

Technologies utilisées

Vous trouverez dans cette section l'ensemble des technologies employées pour la réalisation de notre projet. Elles ont en grande partie été imposées par le sujet. On appelle "technologie" dans ce contexte pour parler de frameworks, d'outil informatique, supportant certaines fonctionnalités (par exemple HyperSQL est la technologie utilisée pour supporter notre base de données).

Technologies Imposées

Backend :

- **Quarkus** (mode natif)
- **JDBI**
- **HyperSQL (HSQLDB)**

Frontend :

- **SolidJS**
- **Bulma**

Technologies Complémentaires

En complément des technologies imposées, nous avons fait des choix pour répondre à des besoins spécifiques. Ces décisions, bien que prises dans des délais souvent contraints, reposent sur une analyse rapide mais justifiée. Nous avons jugé utile de les documenter ici pour offrir une vision complète des solutions mises en œuvre dans le projet.

Connexion du Frontend – Quarkus Quinoa

Technologie	Description	Avantages	Inconvénients	Pertinence pour SolidJS
Quarkus Quinoa	Intégration native de Quarkus pour gérer les projets frontend avec des outils JavaScript modernes (e.g Vite)	- Configuration minimale (4 lignes) - Gestion transparente des dépendances NPM. Compatible avec Vite utilisé par SolidJs	Dépendant de Node.js pour le build initial (sauf que dans les consignes du projet, on a le droit de faire ça)	Idéal : Simplifié le développement avec SolidJS, aligné avec les meilleures pratiques modernes.
Static Server	Hébergement direct de fichier HTML/CSS dans le répertoire <code>src/main/resources</code>	- Facile à configurer. - Aucun outil externe requis pour le build.	- Pas de support natif pour les frameworks modernes comme SolidJS. - Build manuel nécessaire.	Faible : Non adapté pour SolidJS, car ne gère pas les dépendances modernes comme Vite.
Build Frontend Manuel	Utilisation d'un outil de bundling tiers produire les fichiers frontend	- Facile à configurer. - Aucun outil externe requis pour le build.	- Nécessite une configuration manuelle. - Complexité accrue pour les nouveaux utilisateurs.	Acceptable : Fonctionne avec SolidJS mais nécessite une configuration spécifique à Vite.

Conclusion : Nous avons choisi Quarkus Quinoa pour sa compatibilité native avec Quarkus et sa simplicité d'intégration (4 lignes dans notre application.properties)

Pour plus ou moins d'info : [Quarkus for the Web](#)

Streaming de Données – WebSocket

Technologie	Avantages	Inconvénients	Pertinence pour notre projet
WebSocket	Bidirectionnel, faible latence, intégration native dans Quarkus.	On a pas eu de problème, la doc est claire.	Idéal : Mises à jour dynamiques en temps réel.
Kafka	Scalable, résilient, persistant, support Quarkus solide.	On ne sait pas mais pas adapté pour notre projet	Indirect : Bon pour backend, moins direct pour SolidJS.
SSE	Simple, léger pour les flux unidirectionnels.	Pas de communication client-serveur bidirectionnelle.	Inacceptable : Limité pour des interactions complexes.

Conclusion : Nous avons choisi **WebSocket** en raison de sa **communication bidirectionnelle** en temps réel, essentielle pour envoyer à la fois du **client vers le serveur** (ex. : code utilisateur, erreurs de compilation) et du **serveur vers le client**. Contrairement à **SSE**, qui est **unidirectionnel**, WebSocket répond mieux à nos besoins interactifs (nous n'avons pas testé SSE sur notre projet directement à cause du fait qu'il soit **unidirectionnel**). Bien que **Kafka** soit puissant, sa **complexité** ne correspondait pas à notre projet.

Gestion de la Base de Données – Agroal

Critère	Agroal	HikariCP
Intégration avec Quarkus	+ Pré-intégré	- Besoin de configuration
Performance	+ Très performant	+ Performant
Documentation/Support	+ Documenté pour Quarkus	= Standard

Conclusion : Agroal est le choix naturel pour notre projet Quarkus, car il est intégré nativement et offre de bonnes performances sans configurations supplémentaires.

Finalement, face à la simplicité et aux bonnes performances d'Agroal, nous avons décidé de l'adopter définitivement pour notre gestion des connexions dans Quarkus.

Nous avons testé **HikariCP** dans un premier temps, mais avons rencontré plusieurs difficultés :

1. **Configuration complexe** : HikariCP nécessite un nombre important de configurations spécifiques, ce qui a ajouté une complexité inutile au projet.
2. **Problèmes de fonctionnalité** : Malgré nos efforts pour configurer HikariCP correctement, nous avons rencontré des problèmes techniques qui ont rendu son utilisation non fonctionnelle dans notre contexte.

Téléchargement des modèles – Maven-antrun-plugin

Critère	Maven-antrun-plugin	googlecode.maven-download
Flexibilité	+ Hautement flexible	= Bonne flexibilité
Documentation/Support	= Standard	+ Bien documenté
Complexité de configuration	+ Configuration simple	- Plus de configurations

Conclusion : Bien que plus complexe à configurer, le maven-antrun-plugin offre une flexibilité supérieure pour nos besoins spécifiques de téléchargement de modèles. Nous avons utilisé **googlecode.maven-download-plugin** au début car simple à configurer en utilisant juste wget et des arguments sauf qu'on a pas trouvé un moyen de télécharger les modèles en parallèle, le téléchargement se fait un par un c'est principalement ce qui a justifié notre choix pour **Maven-antrun-plugin**.

Messages LLM côté Front – Prisms et Marked

Critère	Prism + Marked	Highlight.js + CodeMirror
Fonctionnalité principale	Syntaxe et rendu Markdown simple et rapide avec mise en surbrillance.	Éditeur de code interactif avec syntaxe enrichie.
Légèreté	Très léger	Plus lourd (CodeMirror ajoute une surcharge)
Compatibilité avec SolidJS	Bonne compatibilité grâce à des bibliothèques simples et légères.	Peut nécessiter des wrappers ou des adaptations pour fonctionner correctement.

Conclusion : Notre llm renvoie du format Markdown – Prisms + Marked est une solution plus légère, plus simple à intégrer et mieux adaptée que Highlight.js + CodeMirror, qui offre des fonctionnalités superflues pour cet usage.

Répartition des tâches

Nous détaillerons ici la répartition des tâches au cours de la réalisation du projet au sein du binôme. Il est à rappeler que la grande majorité du projet a été réalisée à deux, ainsi si certaines tâches ont été la responsabilité de l'un ou l'autre, les deux membres sont responsables de la totalité de la production.

Max TEKPA – Backend, communication et direction de production

Bonjour, je suis Max TEKPA et cette section est dédiée à mes responsabilités quant à la production de Xplain. Nous avons beaucoup communiqué, notamment nos professeurs, mais aussi nos camarades, ont répondu aux questions que nous nous posions en passant par moi, mais ce qui reste la meilleure interaction que j'ai pu avoir, ça a été celle avec Jake Luciani, alias Tjake. Principal développeur de Jlama, il a répondu à mes questions personnellement ce qui nous a permis d'avancer plus vite, surtout au début, quand nous ne savions encore rien de l'importation de modèles d'IA dans un projet Maven.

Marc LE COQUIL – Frontend, mode natif et traduction

Je suis Marc LE COQUIL et j'explique dans cette partie quelles étaient mes responsabilités au sein du projet Xplain. J'étais le principal responsable du passage de notre application en mode natif, comme cela sera expliqué plus tard cette tâche m'a pris la grande majorité de mon temps de production. Malheureusement cette partie de mon travail ne se retrouve pas dans la version finale du projet, nous vous expliquons dans la section des difficultés rencontrées pourquoi.

En suivant les directives du sujet et de Max, j'ai aussi participé dans la lecture de la complexe documentation de nos technologies, ce qui m'a notamment permis de trouver la dépendance Quinoa et d'être le premier à proposer un frontend. J'ai donc aussi été celui qui l'a entretenu au fur et à mesure des avancées faites en backend.

Enfin, j'ai grandement participé à la migration de notre code vers l'anglais car plus à l'aise avec la langue. Même si cela peut passer pour un détail, traduire

des mots écrits en français vers l'anglais m'a permis de relire et comprendre chaque parcelle de code et ainsi de comprendre même celles que je n'ai pas réalisées.

Travail à deux – Tests, rédaction des documents et recherche dans la documentation technique

Tous les tests ont été réalisés par le membre n'ayant pas codé le code testé. Ainsi nous avons pu mettre à l'épreuve au mieux notre code avec un regard le plus objectif possible. Cela nous a aussi permis de comprendre le code de notre binôme et donc de ne jamais laisser notre travail rester trop longtemps inconnu l'un de l'autre.

La rédaction du manuel utilisateur et de cette documentation technique a été réalisée par les deux membres en fonction des sujets qu'ils maîtrisaient le mieux. L'esthétique et l'organisation des documents est notamment attribuée à Marc LE COQUIL tandis que la description des technologies est le travail de Max TEKPA, chaque autre paragraphe a été co-écrit.

Le travail le plus chronophage de ce projet ne pouvait être réalisé par un seul membre. Nous avons été tous deux plongés dans la documentation des différentes technologies, en particulier celle de Quarkus et GraalVM. Nous avons sut discriminer les informations utiles ou non en fonction de nos recherches, tout en nous échangeant régulièrement les liens vers ces dernières.

Difficultés rencontrées

Ce projet fut très difficile à réaliser, nous exposons dans cette partie les difficultés rencontrées durant sa production et quelles auront été leurs conséquences sur le résultat final. À commencer par ce qui a pu être réalisé, mais sans oublier ensuite ce qui a été abandonné.

Ce qui a été difficile à réaliser

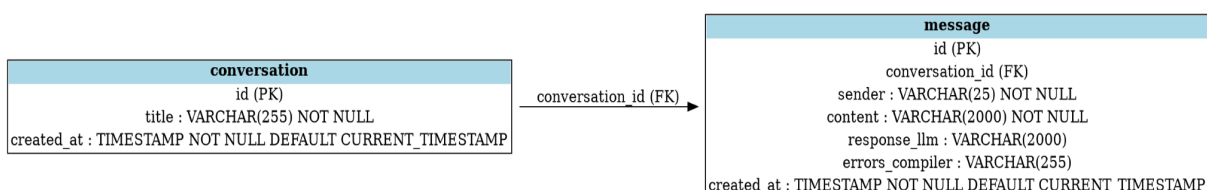
À la soutenance beta, nous avons pu poser des questions par rapport à nos technologies et la bonne utilisation de certains outils. Ainsi nous avons amélioré et ajouté de nouvelles fonctionnalités.

- L'utilisation de Jdbi, avant la soutenance beta, nous n'utilisons pas les bonnes méthodes (Mapper) pour enregistrer des données dans notre base, ce qui rendait le code compliqué et moins agréable à lire, nous l'avons changé avec des interfaces implémentant RowMapper.
- Nous avons eu quelques problèmes avec notre base de données, nous pensons que cela est dû au fait que quarkus-agroal ne supporte pas nativement HYPERSQL, nous avons donc eu dû mal à faire un fichier externe contenant notre bdd.
- Nous avons également dû améliorer notre schéma de la base de données car nos échanges avec le llm n'étaient pas des discussions, c'est ce qu'on nous a fait remarquer à la soutenance beta.

L'ancienne table de la bdd:

demand
id (PK)
sender : VARCHAR(20) NOT NULL
content : VARCHAR(2000) NOT NULL
response_llm : VARCHAR(2000)
errors_compiler : VARCHAR(255)

Les nouvelles tables:



Ce qui n'a put être réalisé

Le projet dans son ensemble a été réalisé en respectant toutes les consignes à l'exception d'une seule : nous ne sommes pas parvenus à développer notre application avec Quarkus Natif. Il s'agit là d'un hors sujet dont nous sommes conscients, mais nous expliquerons ici ce que nous avons fait pour tenter de résoudre ce problème et ce qui nous a amené à abandonner.

Une documentation contradictoire et éclatée

Les technologies imposées pour notre production avaient déjà la particularité d'être assez peu documentées quant à leur liaison, on peut notamment citer l'absence de [guide](#) dans la liaison entre une base de données HyperSQL et un serveur Quarkus, nous sommes parvenu à résoudre le problème à l'aide d'un [driver](#) mais ce dernier ne permettait pas une bonne transition vers le mode natif. Et ce problème ne se limitait pas à la base de données, on le retrouvait aussi dans le téléchargement des LLM ou dans la communication avec l'API REST. À chaque fois la documentation se contente d'indiquer ce qu'il faut faire dans le cas normal, puis de rediriger vers une autre documentation pour le mode natif, qui elle-même va indiquer des contradictions avec les précédentes consignes.

Des erreurs non documentées ou aux solutions non fonctionnelles

Car en effet il y a plusieurs façon de faire une application utilisant Quarkus en mode natif:

La [documentation Quarkus](#) dit d'utiliser la commande suivante

```
./mvnw install -Dnative
```

Le Readme.md créé avec le projet Maven propose plutôt:

```
./mvnw package -Dnative
```

Mais bien sûr Maven a sa [propre commande](#) :

```
./mvn package -Pnative
```

Cette dernière permet d'éviter d'utiliser la commande native-image de GraalVM directement, cependant une grande partie de la [documentation](#) disponible ne concerne que ce dernier. Nous nous sommes perdu, ne sachant pas qui écouter, que tester, que prioriser, comment modifier notre pom.xml. Car si la documentation précédente indique un profil semblable à celui ci:

```
<profiles>
  <profile>
    <id>native</id>
    <build>
      <plugins>
        <plugin>
          <groupId>org.graalvm.buildtools</groupId>
          <artifactId>native-maven-plugin</artifactId>
          <version>${native.maven.plugin.version}</version>
          <extensions>true</extensions>
          <executions>
            <execution>
              <id>build-native</id>
              <goals>
                <goal>compile-no-fork</goal>
              </goals>
              <phase>package</phase>
            </execution>
            <execution>
              <id>test-native</id>
              <goals>
                <goal>test</goal>
              </goals>
              <phase>test</phase>
            </execution>
          </executions>
        </plugin>
      </plugins>
    </build>
  </profile>
</profiles>
```


La [documentation Maven](#) ajoute des configurations qui ne précisent ni où les placer, ni si cela va créer des conflits avec autre chose (ça le fera). Et la [documentation Quarkus](#) propose son propre profile natif beaucoup plus simple :

```
<profiles>
  <profile>
    <id>native</id>
    <activation>
      <property>
        <name>native</name>
      </property>
    </activation>
    <properties>
      <skipITs>false</skipITs>
      <quarkus.native.enabled>true</quarkus.native.enabled>
    </properties>
  </profile>
</profiles>
```

Tout en utilisant Maven comme outil de montage, cette documentation ne parle que la commande native-image. Y a t il une adaptation à faire pour le plugin Maven ? Sommes nous obligés d'utiliser la commande native-image car plus documentée ? Y a t il quelque chose que nous n'avons pas vu ? Peut être en cherchant plus, plus longtemps nous trouverons.

Un apprentissage sur le tas

Car oui, nous ne savons pas. Le sujet implicite de ce projet est l'apprentissage sur le tas de technologies et de méthodologies **pendant** le développement. Cependant cet apprentissage libre a le défaut que rencontrent les autodidactes : celui de ne pas savoir où chercher. Car si toutes informations présentes dans une doc est fiable car écrite par les développeurs, cela ne garantit pas sa compatibilité avec les autres docs, et encore moins son utilité pour le projet en cours. Nous en avons fait les frais en testant moult outils, commandes, combinaisons, astuces plus ou moins bancales dont le nombre ne cessait de croître.

Si en plus on ajoute à la consigne l'interdiction d'utiliser d'autre technologies que celle spécifiées dans l'énoncé, prenons l'exemple des tests

uniquement acceptés avec Junit5 quand Mockito est largement préférée pour les tests sur les bases de données, on se met même à refuser les seules solutions proposées par certaines documentations. Devant contourner ces problèmes comme avec le driver pour HyperSQL alors qu'un développeur plus libre aurait simplement choisi d'utiliser H2 qui partage de grande similitude et qui, elle, a sa liaison à Quarkus documentée.

Des délais trop courts

Enfin, nous aurions pu y arriver si l'on avait eu peut être 6 semaines de productions accordées en plus. En effet, les deux membres du binôme ont fait partie du Last Project qui, à lui seul, leur a pris plus d'un tiers, si ce n'est la moitié, de leur temps d'étude normal. Ajoutons à cela l'alternance de Max TEKPA, les contretemps médicaux de Marc LE COQUIL en début de semestre, l'impossibilité de travailler à l'université à cause des crash fréquents des postes, du manque de ressources hardware ou l'inaccès à certaines commandes comme le "sudo" parfois indispensables, les problèmes rencontrés sur nos ordinateurs personnels (voir [Annexe](#)) et enfin le fait que nous soyons le seul binôme de la promotion à utiliser Quarkus natif pour notre projet (impossible donc de demander de l'aide ou une documentation quelconque à un autre binôme et impossible de nous fier à d'autres pom.xml, commande de compilation, ou code tout court). En prenant tout ça en compte, nous avons fini par faire un choix.

Et ce choix fut celui d'abandonner le mode natif, quitte à risquer le hors sujet. En effet, nous ne pouvions pas risquer de ne pas coder notre projet simplement parce que nous n'arrivions pas à faire une seule chose, cette dernière n'étant même pas bloquante car un projet correctement codé en mode développement doit pouvoir être monté en un exécutable natif sans mal. Marc LE COQUIL a fini par concentrer son temps sur le passage au mode natif, à commencer par comprendre chaque détail le concernant, tandis que Max TEKPA produisait le backend en parallèle de sorte à ne pas être bloqué. Malheureusement, quand le développement touchait à sa fin, Marc n'avait alors que trop peu participé à la programmation même et le mode natif n'était toujours pas viable. Ainsi, pour éviter une trop grande disparité dans les quantités de travail présentées dans le résultat final, le mode natif a été définitivement

oublié. Nous comptons sur votre compréhension et serons tout disposés à répondre à vos interrogations lors de la soutenance.

Les bugs restants

- Si le navigateur utilisé est firefox on ne peut pas valider une édition de nom de conversation avec "entrer"
- Lorsque l'on cherche à renommer une conversation et que l'on met un nom trop long, on obtient une erreur côté serveur.
- Lorsque plusieurs utilisateurs parlent sur la même conversation, l'application ne plante pas mais les messages se chevauchent en séparant mal les interactions.

Conclusion

Nous pouvons maintenant conclure cette documentation technique. Nous aimerions préciser que si nous n'avions pas tous deux participé au Last Project, le projet Xplain aurait été le plus gros projet de notre parcours universitaire jusqu'à présent. Il nous aura appris combien il est important de bien choisir les frameworks permettant la réalisation d'un projet, qu'il faut savoir s'adapter, apprendre sur le tas et revenir sur ses erreurs. Nous en sortons grandis, reconnaissant pour cette opportunité de nous rapprocher du métier de développeur full-stack et nous espérons que notre travail sera apprécié autant que nous avons apprécié le produire.

Nous vous remercions pour la lecture de cette documentation technique.

Ordinateur de Max TEKPA le 27/12/2024:

[illegible]

Les messages d'erreurs illisibles lors du développement du mode natif:

[illegible]